CSCI 5144 Coding Assignment 1
Team Members: Amina Razzaque, Joshua Lunde, Matthew Gaasedelen

Design Decisions and Justification

We implemented a single-server key-value store in C using POSIX sockets for HTTP communication and POSIX threads to handle concurrent client requests. For storage, we used a linked list in-memory storage because it is straightforward to implement and supports key-value pairs to be inserted, updated, retrieved, and removed. To prevent thread conflicts, we used a mutex to protect shared data during concurrent operations. For persistence, we used an append-only log (data.wal) that records successful PUT and DELETE operations. When the server restarts, it reads from this file to restore the stored data. Lastly, the server has a timestamped operations log and returns appropriate HTTP status codes for successful and unsuccessful requests.

Challenges

One challenge was that since we are working in C, we had to be careful with memory management, making sure memory was properly allocated and freed when values were updated or deleted. Another challenge was ensuring consistency when multiple clients accessed the key-value store at the same time. We addressed this by using a shared global mutex on the store to prevent operations from interfering with each other. A final challenge was that we had to make sure our persistence system handled both PUT and DELETE operations so deleted keys wouldn't reappear after restarting the server.

Assumptions

We assume that the user will know that the code accepts JSON objects in the form of {"value":"some string"). We also assume that these requests are written in JSON, as they are parsed as such no matter what. Another assumption made regarding the requests was the length of the values, which can be no more than 4096 characters. It is also assumed that socket reads and writes will take less than five seconds, as they time out after that. In addition, requests submitted by the user are HTTP/1.x and no more than 16 KB. The program also has no authentication for someone trying to connect to the server, so we assume that users are trusted.

Potential Improvements

For next steps in improving our single-server key-value store, we can implement a hash table to store our key-value pairs. Rather than having our code go through every entry in the linked list, a hash table would allow the key-value store to run in O(1) time when retrieving the value. Another potential improvement would be to prevent duplicate key-value pairs from being written in the persistent memory. Currently, whenever the user runs the 'PUT' function, the pair gets written into our persistent memory, and the most recently inserted key-value will be outputted when the user calls the 'GET' function. The issue arises when the user runs the 'PUT' function multiple times on the same key-value pair. The number of times it's run is the number of times it will be saved off in the persistent data file. Rather than saving off the same thing multiple times, it would be more memory efficient to only save modifications and new pairs.